

Multi-RCM Kinematic Control for a ROSA-like Surgical Robot

A Unity Simulation of Double Remote-Center-of-Motion Control

Samuele Civale
Francesca Forghieri
Alexandru Vivian Pita

Abstract

This report presents a Unity-based simulation of a ROSA-like six-degree-of-freedom surgical robot implementing double Remote-Center-of-Motion (RCM) control, following the task-priority formulation of Aghakhani et al. (ICRA 2013). A from-scratch damped-least-squares controller with a null-space secondary objective handles four operating modes and a three-phase safe-insertion sequence. A Task 2 failure caused by wrist-joint saturation was resolved with strict tip-first hierarchy, an active-set constraint on joint 4, adaptive damping, and bounded integration substeps. The updated controller is evaluated on a 2696-sample Unity log using clinically stricter thresholds of 2 mm for entry-RCM stability and 3 mm for tip-target accuracy. All nine automatic validation checks pass, including zero skull/corridor violation.

Contents

1	Introduction and Clinical Motivation	3
2	The Remote-Center-of-Motion Concept	3
3	System Description	3
3.1	Simulated Manipulator	3
3.2	Task Modes	4
3.3	Task 3: The Safe-Insertion Sequence	4
4	Controller Architecture	5
4.1	Task-Priority Formulation	5
4.2	Task 2 Failure Analysis and Controller Fix	5
5	Experimental Methodology	6
5.1	Data Source	6
5.2	Data Integrity Verification	6
5.3	Validation Checks	6
6	Results	6
6.1	Task Timeline	6
6.2	Overall Validation Summary	7
6.3	Task 3: Safe Insertion	8
6.4	Task 2: Target-RCM with Entry Cone	10
6.5	Task 4: Entry-RCM with Tip Cone	10
6.6	Kinematic and Runtime Behavior	11
7	Limitations	12
8	Conclusion	13

1 Introduction and Clinical Motivation

Stereotactic neurosurgical procedures — biopsy, electrode placement, catheter insertion — require a rigid tool to reach a deep target through a single, pre-planned skull opening. Once the entry point is fixed, any lateral force exerted by the tool shaft against the edges of that opening risks tearing or bruising healthy tissue along the corridor. Robotic assistance can enforce this constraint far more precisely than a hand-guided or passively braced instrument: the robot’s controller can be made to guarantee that the tool axis always intersects the entry point, regardless of how the arm reconfigures to reach different targets.

The mechanism that formalizes this guarantee is the *Remote Center of Motion* (RCM): a virtual pivot point, external to the robot’s own mechanical structure, through which the tool shaft is constrained to pass at all times. Unlike a mechanical RCM (a physical gimbal, as used in some laparoscopic robots), the RCM studied here is purely a *control* objective, implemented as an additional task in a task-priority kinematic controller.

This project implements and validates such a controller in Unity, for a ROSA-like six-degree-of-freedom manipulator, following the double-RCM task formulation of Aghakhani et al. (ICRA 2013), in which the RCM constraint can be located either at the entry point or, in an alternate mode, effectively relocated to the target itself.

2 The Remote-Center-of-Motion Concept

The RCM point is expressed as a point sliding along the tool shaft:

$$p_{\text{RCM}}(q, \lambda) = p_{\text{base}}(q) + \lambda L z(q), \quad (1)$$

where $p_{\text{base}}(q)$ is the position of the tool’s proximal reference point (a function of the joint configuration q), $z(q)$ is the tool’s axial unit vector, L is the tool length, and $\lambda \in [0.02, 0.995]$ is a dimensionless *penetration coordinate* that slides the candidate RCM point along the shaft as insertion proceeds. The controller drives λ together with the joint velocities so that $p_{\text{RCM}}(q, \lambda)$ coincides with the desired entry (or target) marker, without requiring any physical hinge at that location.

Enforcing this constraint changes the character of the motion completely. Without it, nothing prevents the shaft from levering sideways against the entry point as the arm reconfigures to track a moving target. With the RCM task stacked alongside the tip task, the controller can move the tip to a new target while keeping the pivot locked at the entry, or — in the alternate task modes studied here — relocate the effective pivot to the target while the entry side sweeps a bounded cone. This is the “double RCM” idea referenced in the title: the same controller structure realizes an RCM at the entry *or* at the target, selected by which task mode is active.

3 System Description

3.1 Simulated Manipulator

The manipulator is a procedurally generated six-degree-of-freedom serial arm with a ROSA-like (plausible, not vendor-measured) Denavit–Hartenberg table, summarized in Table 1. Joint limits are symmetric except for the last joint (wrist roll), which is effectively unbounded ($\pm 360^\circ$).

Table 1: Denavit–Hartenberg parameters used for the procedural arm (as implemented in `ROSADoubleRCMController.cs`).

Link	a [m]	α [deg]	d [m]	θ_{offset} [deg]	Joint range [deg]
1	0.045	−90	0.42	0	[−170, 170]
2	0.42	0	0.00	0	[−120, 120]
3	0.055	−90	0.00	0	[−150, 150]
4	0.000	90	0.36	0	[−170, 170]
5	0.000	−90	0.00	0	[−120, 120]
6	0.000	0	0.12	0	[−360, 360]

The scene, arm, needle tool, and target/entry markers are all built procedurally at runtime by a single script (`Project4SceneBuilder.cs`), rather than hand-assembled in the Unity editor, keeping the project self-contained and reproducible. A second script (`FreeFlyCameraKeyboard.cs`) provides an independent keyboard fly-camera for visual inspection and screen recording; it has no effect on the controller or the logged data.

3.2 Task Modes

All logic — kinematics, the control law, the insertion state machine, the on-screen overlay, and CSV logging — lives in a single controller script, `ROSADoubleRCMController.cs`. It exposes five mutually exclusive modes, four of which are active task modes (Table 2); the fifth, `Hold`, freezes the arm and is not part of the validated task set.

Table 2: Task modes implemented in the controller.

Task	Enum name	Description
T1	<code>EntryRCM_TipTarget</code>	RCM locked at the entry point while the tip tracks a fixed target.
T2	<code>TargetRCM_EntryCone</code>	RCM relocated to the target while the entry side sweeps a bounded cone.
T3	<code>InsertionSequence</code>	Three-phase safe-insertion sequence (the main clinical task, Section 3.3).
T4	<code>EntryRCM_TipConeAroundTarget</code>	RCM locked at the entry while the tip sweeps a bounded cone around the target.

3.3 Task 3: The Safe-Insertion Sequence

Task 3 is a three-phase finite-state machine that mirrors the intended clinical workflow (a fourth internal state, `Done`, marks completion):

1. **ApproachEntry** — the needle axis is aligned with the entry–target line while the tip approaches from outside the trocar; the RCM constraint is *not yet* active.
2. **PierceEntry** — the tip travels from the outside approach point to the entry (trocar) point.
3. **InsertToTarget** — the RCM is locked at the entry point and the tip advances along the shaft axis to the target, terminating in the `Done` state once both the insertion progress and the tip-to-target distance fall below fixed thresholds.

4 Controller Architecture

4.1 Task-Priority Formulation

At every control step, the active point tasks (tip position, entry/target cone tracking, and the RCM point) are stacked into a single differential kinematics problem:

$$\begin{bmatrix} \dot{x}_{\text{task}} \\ \dot{x}_{\text{RCM}} \end{bmatrix} = \begin{bmatrix} J_{\text{task}} & 0 \\ J_{\text{RCM}} & \end{bmatrix} \begin{bmatrix} \dot{q} \\ \dot{\lambda} \end{bmatrix}, \quad (2)$$

where each row block is itself the vertical stack of whichever point tasks are active in the current mode. Each task’s desired velocity is a saturated proportional term on its Cartesian error,

$$\dot{y}_i = K_i (x_{\text{des},i} - x_i), \quad \text{saturated to a maximum magnitude}, \quad (3)$$

with per-task gains $K_{\text{tip}} = 3.5$, $K_{\text{RCM}} = 17.0$, and $K_{\text{cone}} = 6.0$ (as configured in `ROSADoubleRCMController.cs`). The comparatively high RCM gain reflects the priority given to the safety constraint over tracking accuracy.

The stacked system is solved with a damped least-squares (Levenberg–Marquardt-style) pseudoinverse,

$$\begin{bmatrix} \dot{q} \\ \dot{\lambda} \end{bmatrix} = J^T (J J^T + \mu^2 I)^{-1} \dot{y}, \quad \mu = 0.055, \quad (4)$$

implemented from scratch with a custom Gauss–Jordan linear solver; no external inverse-kinematics or optimization library is used. Jacobians are computed by numerical finite differencing rather than symbolically, a deliberate simplicity trade-off that keeps the Denavit–Hartenberg table easily replaceable at the cost of some numerical noise.

A null-space secondary objective is projected into the kernel of J and added on top of the primary solution,

$$\begin{bmatrix} \dot{q} \\ \dot{\lambda} \end{bmatrix}_{\text{total}} = \begin{bmatrix} \dot{q} \\ \dot{\lambda} \end{bmatrix} + s (I - J^+ J) w, \quad (5)$$

with null-space scale $s = 0.55$ and w combining three sub-objectives, each with its own gain: keeping λ near a nominal target (gain 0.9), keeping joints away from their limits (gain 0.08), and biasing the arm away from the skull surface (gain 1.3). The damped projector minimizes interference with primary tasks; Task 2 additionally suppresses the q4 null-space component whenever its active-set constraint is engaged.

Finally, both joint and λ velocities are saturated ($\dot{q}_{\text{max}} = 80^\circ/\text{s}$, $\dot{\lambda}_{\text{max}} = 0.55 \text{ s}^{-1}$) before Euler integration:

$$q_{k+1} = \text{clamp}(q_k + \dot{q} \Delta t), \quad \lambda_{k+1} = \text{clamp}(\lambda_k + \dot{\lambda} \Delta t, 0.02, 0.995). \quad (6)$$

4.2 Task 2 Failure Analysis and Controller Fix

In the original Task 2 implementation, the tip-target and moving entry-cone tasks were solved as one weighted stack. Near a rank-deficient two-point configuration, the solver could select a wrist branch that drove joint 4 toward its -170° limit. Once the physical clamp became active, the saturated joint command no longer represented the Cartesian direction returned by the inverse-kinematics solve. The result was drift of the tip away from its fixed target even though the requested pose remained reachable. Increasing gains did not address the cause and could amplify the instability.

The corrected controller uses the following compact sequence:

1. solve tip tracking as the primary Task 2 objective;
2. project entry-cone tracking into the damped null space of the tip Jacobian;

3. when joint 4 enters a 25° soft margin and its command points farther outward, remove its Jacobian column, set $\dot{q}_4 = 0$, and redistribute the Cartesian command over the remaining joints and λ ;
4. if the unsaturated solution would exceed the joint-speed limit, repeat the solve with increased damping;
5. split long Unity frames into integration substeps no larger than 8 ms.

The active set is directional: joint 4 is free to move inward and is constrained only when it would move farther toward the physical limit. Null-space motion on joint 4 is also suppressed while this constraint is active. In the July 14 run (reduced-scale geometry, Section 5.1), the Task 2 active set was not engaged at all (0 of 2696 logged samples): the smaller entry/target/needle-length configuration keeps joint 4 well clear of its soft limit throughout, and the tip-target error remained below 0.01 mm. The mechanism itself is unchanged and still available for configurations that do approach the limit. The final implementation retains only these functional mechanisms; temporary A/B switches and Jacobian diagnostics used during investigation were removed.

5 Experimental Methodology

5.1 Data Source

The quantitative evaluation is based on the Unity log `project4_multircm_20260714_193548_422.csv`. It contains 2696 samples spanning 157.19 s and covers all four task modes in the order $T3 \rightarrow T1 \rightarrow T2 \rightarrow T4$. Every quantitative result and figure in this report is derived exclusively from this log.

5.2 Data Integrity Verification

All figures and tables were regenerated from the selected log alone with:

```
python3 analyze_rcm_logs_clean.py <log> --out RCM_analysis_v2
--entry-threshold-mm 2.0 --tip-threshold-mm 3.0 --settle-time-s 1.0
```

The generated figures are stored in `report/generated_figures/july14/`; the corresponding combined data, phase summary, checks, and analyzer notes are stored in `report/generated_data/july14/`. Previous report datasets were left untouched.

5.3 Validation Checks

Nine pass/fail checks are computed with a 1 s settling exclusion after each relevant task or phase transition. The thresholds are: entry-RCM stability ≤ 2 mm, tip-target tracking ≤ 3 mm, entry-/tip-cone tracking ≤ 50 mm, and skull/corridor violation ≤ 0 mm. Raw phase maxima are still reported in the plots and discussion; therefore they can be higher than the settled values used by the automatic checks.

6 Results

6.1 Task Timeline

Figure 1 shows the sequence and duration of the four task modes. The run begins with safe insertion (T3, approximately 0.1–26.8 s), followed by the static entry-RCM/tip-target mode (T1, 26.8–30.8 s), target-RCM/entry-cone tracking (T2, 30.8–95.3 s), and entry-RCM/tip-cone tracking (T4, 95.3–157.3 s).

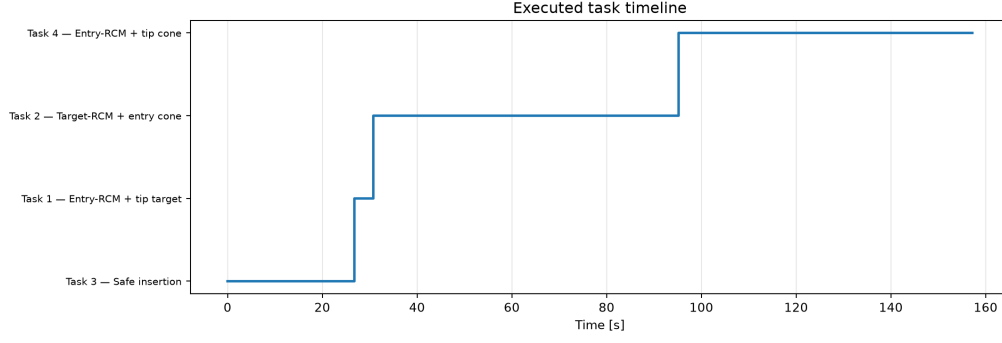


Figure 1: Executed task and phase timeline. Results obtained from the logged experimental run.

6.2 Overall Validation Summary

Table 3 reports all nine checks on the logged run.

Table 3: Validation checks.

Check	Threshold [mm]	Max value [mm]	Result
T1 entry-RCM stability	2	0.083	PASS
T1 tip reaches target	3	0.382	PASS
T2 entry-cone tracking	50	13.665	PASS
T2 tip fixed at target	3	0.009	PASS
T3 insertion entry-RCM	2	0.237	PASS
T3 final target error	3	1.498	PASS
T3 skull/corridor safety	0	0.000	PASS
T4 entry-RCM stability	2	0.834	PASS
T4 tip-cone tracking	50	7.109	PASS

All nine checks pass. The closest margins, relative to their own threshold, are the Task 3 final target error (1.498 mm against 3 mm, 50%) and Task 4 settled entry-RCM error (0.834 mm against 2 mm, 42%). Figure 2 shows every metric normalized by its own threshold, avoiding a misleading plot in which errors with different meanings and scales are overlaid.

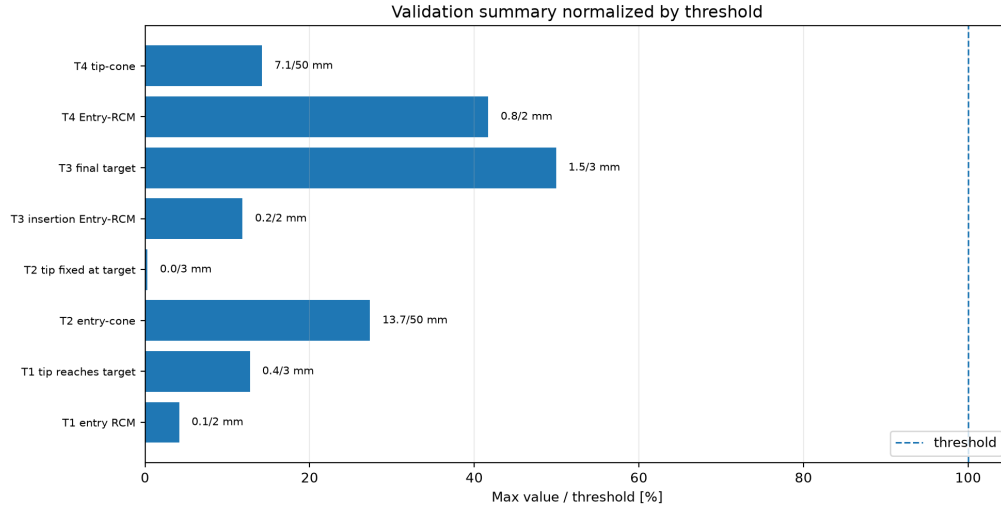


Figure 2: Validation summary normalized by threshold. Results obtained from the logged experimental run.

Figure 3 complements the summary with only the metric that is active in each interval. This task-aware view replaces the earlier all-errors plot, whose simultaneous curves mixed inactive geometric distances with actual tracking errors.

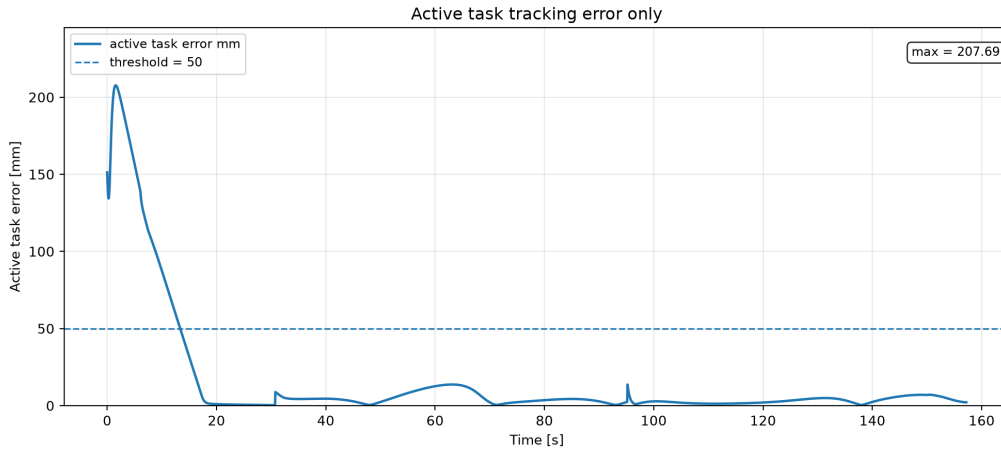


Figure 3: Active task error only; each interval is interpreted using the corresponding task objective.

6.3 Task 3: Safe Insertion

Figures 4–6 validate the core clinical claim of Task 3: once insertion begins, the tool converges to the entry pivot, reaches the target, and remains inside the planned skull corridor.

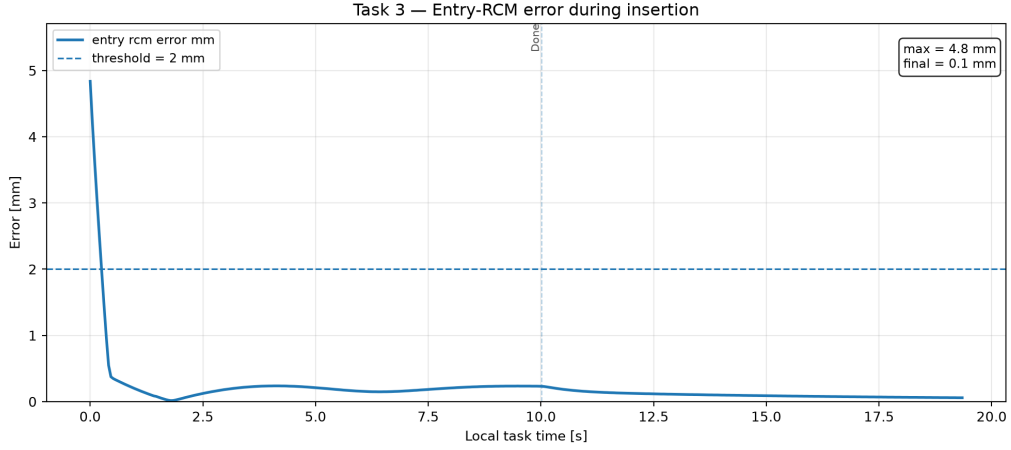


Figure 4: Task 3 entry-RCM error during insertion. The 4.837 mm raw peak is a phase-entry transient; after the 1 s settling interval the maximum is 0.237 mm (2 mm threshold).

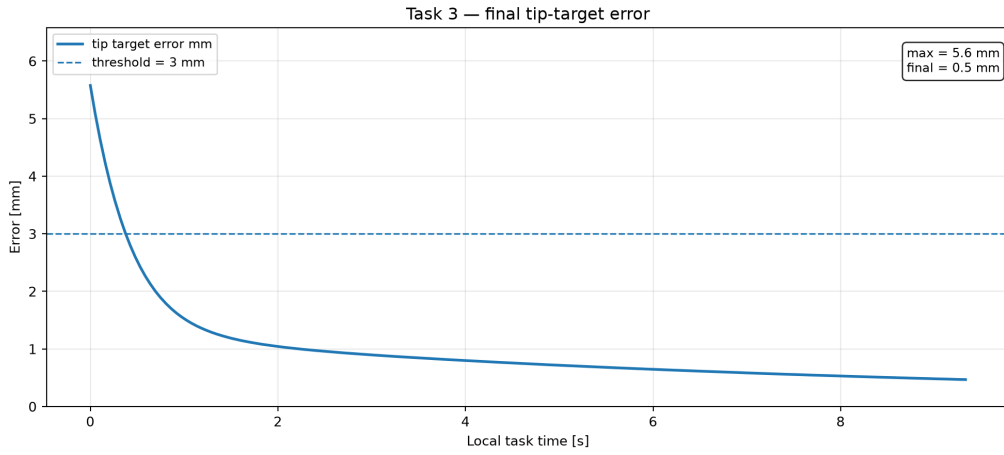


Figure 5: Task 3 tip-to-target error in the `Done` phase. The settled maximum is 1.498 mm and the final recorded error is 0.467 mm (3 mm threshold).

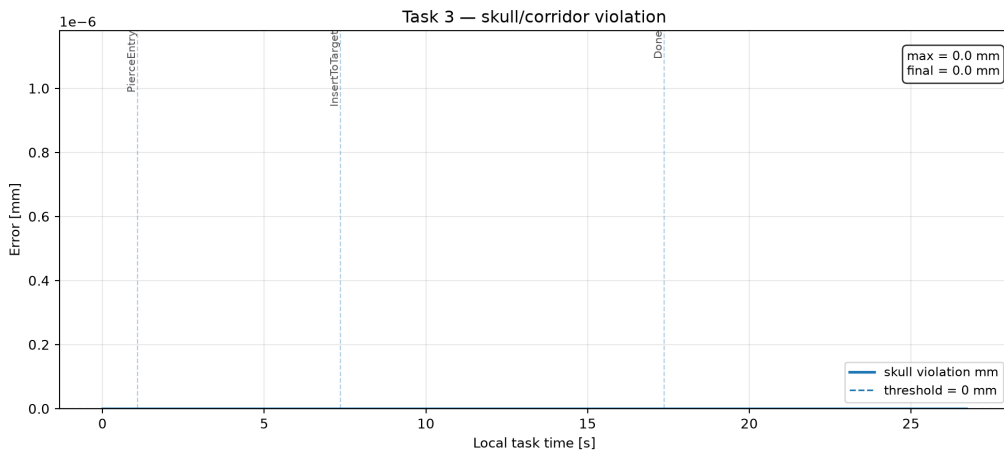


Figure 6: Task 3 skull/corridor violation. No violation is recorded at any time.

The full `InsertToTarget` phase has a raw entry-RCM maximum of 4.837 mm, occurring

immediately after the task switch. Excluding the specified 1 s settling interval reduces the validation maximum to 0.237 mm, and the last insertion sample is 0.232 mm. In the completed state, the target error falls to 0.467 mm. Skull/corridor violation is exactly 0.0 mm throughout.

6.4 Task 2: Target-RCM with Entry Cone

Task 2 keeps the tip fixed at the target while the entry-side point follows a rotating cone. The tip is the strict primary objective and cone motion is secondary, so cone error is allowed to increase when the objectives compete near a joint limit.

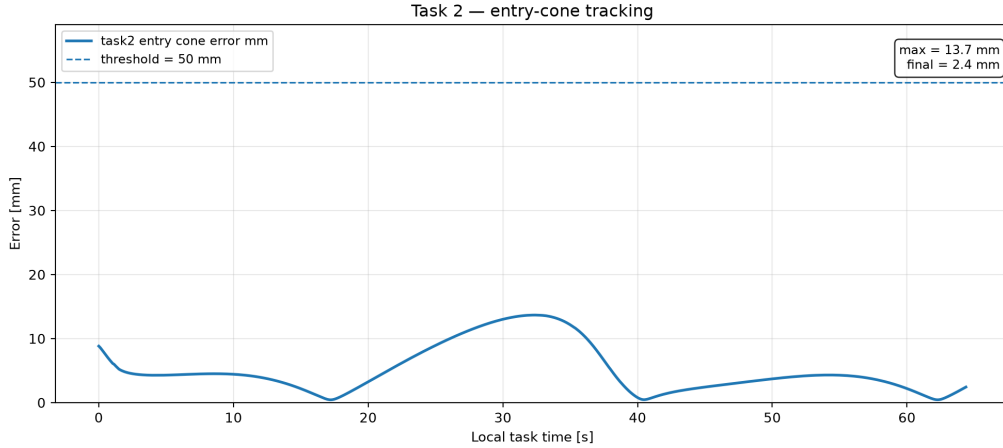


Figure 7: Task 2 entry-cone tracking error: 13.665 mm maximum against the 50 mm threshold.

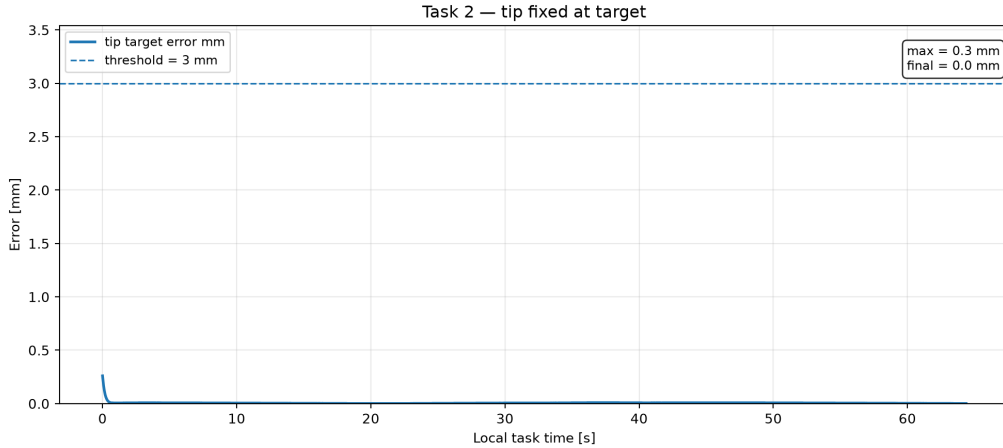


Figure 8: Task 2 primary tip-target error: 0.009 mm maximum against the 3 mm threshold.

These results demonstrate the intended effect of the fix: the tip remains effectively fixed (0.009 mm settled maximum, 0.26 mm raw peak in the pre-settling window), and the secondary entry-cone error remains well within its 50 mm allowance (13.665 mm maximum and 2.428 mm at the end of Task 2). Unlike the earlier, larger-scale run, the Task 2 active set was not engaged at all in this run (Section 4.2), since the rescaled geometry keeps joint 4 away from its soft limit.

6.5 Task 4: Entry-RCM with Tip Cone

Task 4 reverses the geometric emphasis: the entry RCM is primary while the tip traces a cone around the deep target. The raw entry-RCM peak occurs at task activation; after settling, both

validation metrics remain within threshold.

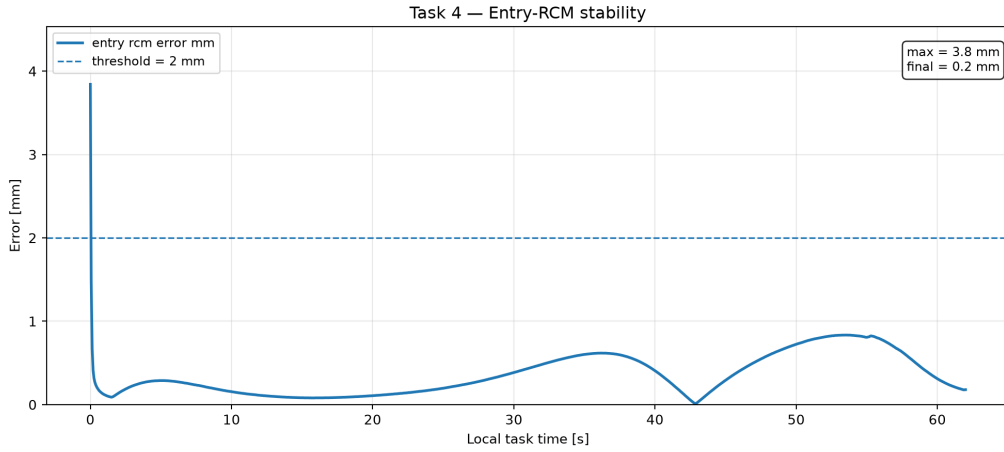


Figure 9: Task 4 entry-RCM error. Raw maximum: 3.842 mm; settled validation maximum: 0.834 mm against 2 mm.

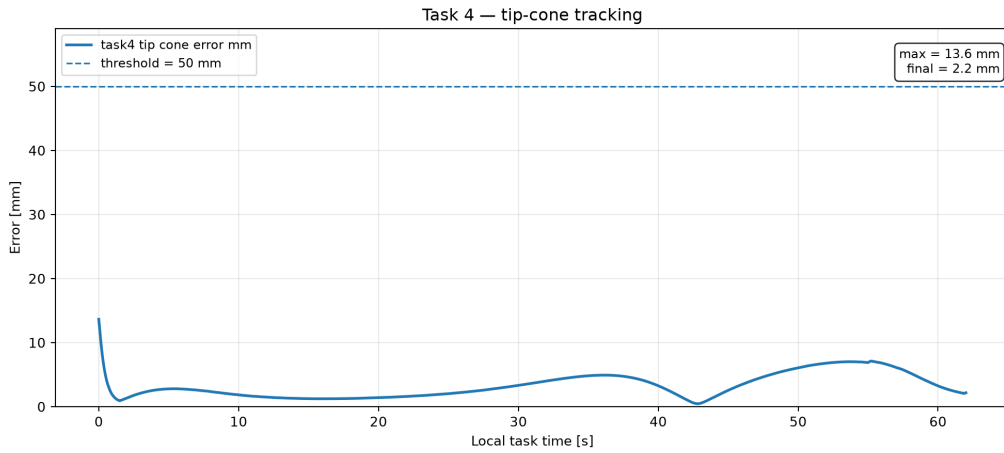


Figure 10: Task 4 tip-cone error. Raw maximum: 13.633 mm; settled validation maximum: 7.109 mm against 50 mm.

6.6 Kinematic and Runtime Behavior

Figure 11 shows the penetration coordinate λ over the full run, ranging from 0.601 to 0.995 within its clamped bounds $[0.02, 0.995]$. Figure 12 shows the joint-velocity norm (peak 1.250 rad/s, mean 0.063 rad/s). The plotted norm combines six joints and is distinct from the $80^\circ/\text{s}$ limit applied to each individual joint.

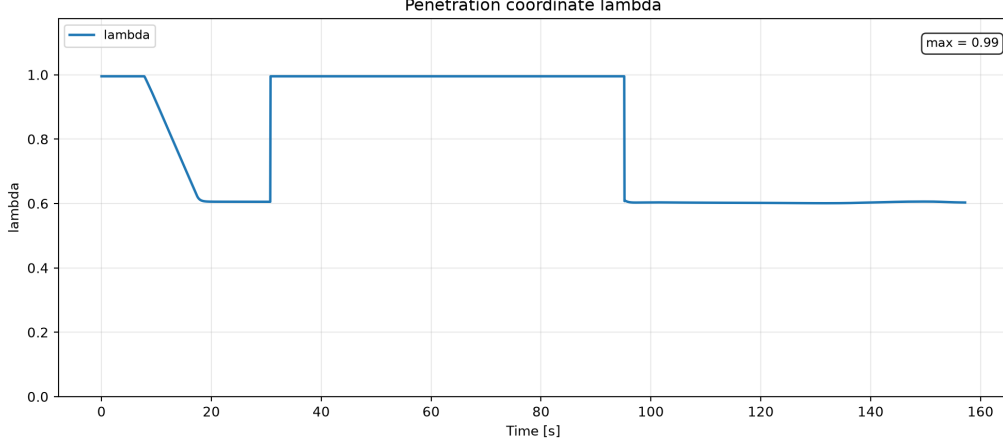


Figure 11: Penetration coordinate λ over the full run. Results obtained from the logged experimental run.

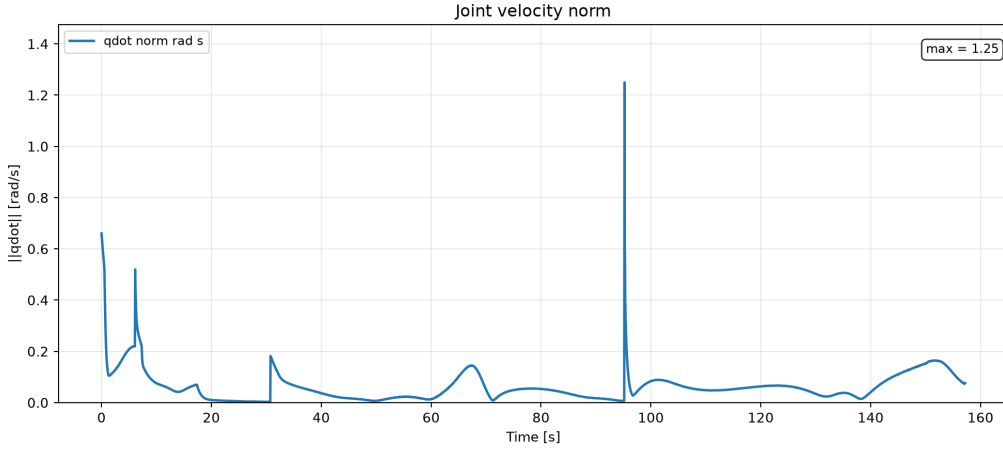


Figure 12: Joint-velocity norm $\|\dot{q}\|$ over the full run. Results obtained from the logged experimental run.

The mean rendering frame rate was 58.4 fps (minimum 5.82 fps, maximum 213.04 fps). Frame-rate variation does not change the validation logic because long frames are subdivided by the controller and the checks are computed from logged geometric state.

7 Limitations

- The manipulator geometry is *ROSA-like*: a plausible Denavit–Hartenberg table, not measured from an official CAD model.
- The needle is modeled as a rigid cylinder; there is no tissue interaction or deformation.
- The controller is purely kinematic: there is no dynamics or force control.
- Jacobians are computed numerically (finite differencing) rather than in closed form, trading some numerical accuracy for implementation simplicity and DH-table flexibility.
- Quantitative conclusions rest on a single logged run, not on repeated randomized trials. Passing all checks demonstrates this execution, not a statistical success rate over the complete workspace.

- The q4 active set is a velocity-level constraint rather than a predictive trajectory optimizer; future configurations may require active constraints on additional joints.

8 Conclusion

This project implements a double-RCM task-priority controller for a simulated ROSA-like surgical robot, covering four task modes and a three-phase clinically motivated insertion sequence entirely from scratch (custom damped-least-squares solve, custom null-space projection, and no external IK library). The updated 2696-sample run passes all nine automatic checks using 2 mm entry-RCM and 3 mm tip-target thresholds, while recording zero skull/corridor violation. In particular, Task 2 holds the tip within 0.009 mm despite its moving cone objective, confirming that the strict hierarchy keeps the previously observed wrist-limit drift from recurring (the q4 active set was not even engaged in this run). Task 4 also passes after settling, with 0.834 mm maximum entry-RCM error. These results validate the implemented kinematic behavior for this run; they do not constitute clinical validation of the ROSA platform or a statistical workspace-wide guarantee.

Future Work

1. Replace the finite-difference Jacobian with an analytic one for speed and numerical robustness.
2. Re-introduce anatomical constraints as an explicitly separate validation layer, rather than mixing them back into the core RCM logic.
3. Adopt real ROSA kinematic parameters, if they become accessible.
4. Run randomized multi-trial validation with matched task durations and repeated initial configurations, including configurations near every joint limit.
5. Generalize the q4 active set to a compact multi-joint constrained solver if broader workspace tests expose saturation of other joints.